iOS Terminal Connection Guide - Ingenico Moby5500

The **iOS Terminal Connection Guide** outlines the steps required to integrate and connect the Ingenico Moby5500 terminal via Bluetooth on iOS. This guide will cover the following areas:

- **Prepare Merchant, Terminal & POS Application objects**: Preparing merchant, terminal, and POS application information needed for initialization.
- **Create and Initialize the Terminal Manager Object**: Creating and configuring the terminal manager with the required setup.
- **Bluetooth Discovery and Pairing**: Discovering nearby terminals, selecting a device, and completing LED-based pairing.
- **Response Handling**: Understanding and responding to common connection issues such as permission errors or hardware failures.

Prerequisites

Before you begin integrating the Deluxe POS SDKs, ensure you have the following prerequisites ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - iOS Merchant Authorization & Accounts Guide

ON THIS PAGE

Prerequisites

Setup Requirements

Prepare Merchant, Terminal & POS Application objects

Create and Initialize the Terminal Manager Object

Bluetooth Discovery and Pairing

Response Handling

Setup Requirements

The TerminalManager class handles all terminal connection functionalities and uses a TerminalManagerDelegate protocol object to asynchronously provide updates on various connection statuses throughout the connection process.

Connecting to the Ingenico Moby5500 terminal requires the appropriate Bluetooth permissions to be configured in the client application. Therefore, it is essential to include the necessary Bluetooth usage descriptions in the app's Info.plist file.

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>App needs to connect with the Pos terminal
to perform payments.</string>
<key>NSBluetoothPeripheralUsageDescription</key>
<string>App needs to connect with the Pos terminal
to perform payments.</string>
```

After integrating all the .xcframework files into your project, you'll have access to all the classes provided by the POSSDK framework. To use any of these classes, simply import the POSSDK module into your own class.

```
import PosSDK
```

Prepare Merchant, Terminal & POS Application objects

Define the MerchantInformation and TerminalInformation objects. These objects are required to initialize the TerminalManager object.

Merchant Information:

```
MerchantInformation(
companyId: "",// This must be set to "000005".
mid: "",// This should be mid of your merchant
account.
tid: "",// This Should be tid of your mechant
account.
bankId: "",// This can be left empty as of now.
merchantDBA: "",// This should be dba of your
merchant account.
accountId: "", // This should be accessToken of
the merchant account.
```

```
accessToken of the merchant account
```

authClientAccessToken can be retrieved using your MerchantAuthorizationService class and merchantAccessToken can be retrieved by using the MerchantAccountService class.

Terminal Information:

```
//This is the INGENICO MOBY 5500 bluetooth
connection terminalinformation
class TerminalInformation(
manufacturer = TerminalManufacturer.ingenico,
model = TerminalModel.ingenicoMoby5500,
connectionType = TerminalConnectionType.bluetooth,
terminalAppName = "DlxCrtiOS",//You can keep this
value now
terminalAppVersion =  "0001",//You can keep this
value now
terminalName = "",
firmwareInfo = "",// optional - This will be
ingenico terminal
firmwareinformation
deviceID = ""// optional - This will be ingenico
terminal serial number eg: K0195509
batteryPercentage = "",// optional - This will be
```

If a valid terminalName is provided, the TerminalManager will attempt to connect directly to that specific terminal. Otherwise, it will initiate a Bluetooth discovery phase to detect all nearby Moby devices and return a list of available terminals.

POSApp Information:

```
class POSAppInformation {
name = "" //This will be the client  application
name
version = "" //This will be the client application
version
}
```

Create and Initialize the Terminal Manager Object

Step 1: Create TerminalManager Object

```
var terminalManager: TerminalManager? = nil
terminalManager = TerminalManager(merchantInfo:
merchantInformation, terminalInfo:
terminalInformation, merchantPosAppDetail:
merchantPosAppInformation)
```

Step 2: Implement Terminal Connection Callback

Implement all the TerminalConnectionDelegate protocols in your class to receive various updates from the TerminalManager object.

```
extension ConnectionViewController:
TerminalConnectionDelegate {
func onBluetoothDiscoveryCompleted(devices:
[String]) {
//Handle all available device
}

func onBluetoothDiscoveryFailed(error: String) {
// Handle when bluetooth discovery failed here
}

func onConnectionFailed(connectionStatus:
PosSDK.TerminalConnectionStatus,
                     errorMessage: String) {
//Handle connection failed here
}
```

Bluetooth Discovery and Pairing

Step 1: Call connect() to start the connection process.

The process begins by scanning for nearby Bluetooth devices.

```
terminalManager?.connect()
```

If the terminalName in the TerminalInformation object is left empty, the connect() method will initiate a Bluetooth discovery process to locate all nearby Moby5500 devices. It will then trigger the onBluetoothDiscoveryCompleted(devices: [String]) callback, which returns a list of available Moby5500 terminal names detected in the vicinity.

If a specific terminalName is provided in the TerminalInformation object, the connect() method will still initiate Bluetooth discovery but will automatically select the specified terminal and proceed with pairing—bypassing the need to call connectBluetoothDevice(selectedDeviceName) separately.

Step 2: Discover and Select a Terminal for Connection

When the onBluetoothDiscoveryCompleted(devices: [String]) method is triggered, it provides a list of nearby moby5500

```
override fun onBluetoothDiscoveryCompleted(devices:
[String]) {
// Save and display the list of discovered devices
this.devices = devices
// Show a UI to let the user select a device
}
```

Once the user selects a device from the displayed list, pass the selected device name to the connectBluetoothDevice method to initiate the pairing process.

```
val selectedDevice = "Device_Name" // Replace with
the user-selected device
terminalManager?.connectBluetoothDevice(selectedDev
iceName: selectedDevice)
```

Step 3: Implement LED Pairing

After invoking the connectBluetoothDevice method, the TerminalManager begins the pairing process. For Moby5500 terminals, LED pairing is initiated, and the onDeviceLedConnectionUpdates(_ ledSequence: [TerminalLedStatus]) method is called with LED status updates.

Display LED Statuses:

Use the ledSequence to show the status of the terminal's LEDs (red, yellow, orange, blue) on your app's UI.

User Actions:

Provide the following options for the merchant:

- **Confirm Pairing**: If the LED indications match between the app and terminal.
- **Repeat Pairing**: To restart the LED pairing process.
- **Cancel Pairing**: To abort the pairing and terminate the connection request.

```
func onDeviceLedConnectionUpdates(_ ledSequence:
[PosSDK.TerminalLedStatus]) {
// Display LED pairing statuses and provide user
actions (Confirm, Retry, Cancel).
// In the UI, based on the ledSequence, show the
appropriate ui and display three buttons: Confirm,
Retry, and Cancel.
// When the user selects an action, call the
```

```
// User actions:
terminalManager?.confirmBluetoothLedPairing()  //
Confirm pairing
```

Once a terminal has been successfully paired with your iPhone, future connection attempts will automatically recognize the pairing and skip the pairing process, connecting directly to the terminal.

Response Handling

On Success: When pairing succeeds, the terminal moves to the connected state, and the onConnectionSucceeded() method is called.

On Failure: If the connection fails, the onConnectionFailed() method is triggered with the reason for the failure.

```
override fun onConnectionSucceeded() {
// Terminal successfully connected and ready for
transactions
}

func onConnectionFailed(connectionStatus:
TerminalConnectionStatus, errorMessage: String) {
// Handle connection failure or cancellation
}
```

connectionStatus indicates the last available status of the terminal, helping to identify what exact reason the connection is failed.

Below are some of the possible terminal connection failure statuses:

TerminalConnectionStatus.bluetoothDiscoveryFailed: When no bluetooth devices found or when bluetooth permission is not enabled.

TerminalConnectionStatus.bluetoothDevicesNotFound: This status occurs when no Bluetooth devices are found or when Bluetooth permissions are not enabled. It typically appears when a connection attempt is made during a transaction workflow.

TerminalConnectionStatus.pairingFailed: This status indicates that pairing failed due to unknown hardware issues. While it's an

TerminalConnectionStatus.pairingFailedCheckDevice: This status indicates a pairing failure due to a terminal hardware issue. If this error occurs, the user must manually go to the iPhone's Bluetooth settings, remove the existing pairing, and then try connecting again.

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation